

Operating Systems

A Practical Hand Note for Backend Engineers

Phase 2 — Computer Science Fundamentals • Topic 4

#	Topic	What it covers
1	Processes vs Threads	Address space, PCB/TCB, isolation, IPC
2	Concurrency	Parallelism, race conditions, atomicity, memory model
3	Locks and Mutex	Mutex, semaphore, spinlock, RWLock, condvar, atomics
4	Deadlock	Coffman conditions, prevention, detection, DB deadlocks
5	Scheduling	FCFS, SJF, RR, MLFQ, Linux CFS/EEVDF, real-time
6	Memory Management	Address space layout, allocators, fragmentation
7	Virtual Memory & Paging	Page tables, TLB, faults, replacement, swap, CoW
8	Linux Processes & Context Switching	fork/exec, /proc, signals, switch cost
9	Practice Lab	Hands-on commands and programs to run
10	Rapid Revision + Interview Q&A	One-page recall sheet

Written to be read once, revised in 20 minutes, and practised at a terminal.

How to use this note

This note is written in three layers so it works for both learning and revision. First read the prose to build the mental model, then copy the code and commands into a real Linux box, then use Section 10 as the night-before-interview recall sheet.

- **Prose blocks** — the "why". Read these once, slowly.
- **MENTAL MODEL boxes** — the compressed intuition worth memorising.
- **GOTCHA boxes** — the mistakes that show up in production and interviews.
- **Code / command blocks** — run every one of them. OS concepts do not stick from reading; they stick from watching `pidstat` move.
- **Practice Lab (Section 9)** — graded exercises, from 5-minute to 1-hour.

MENTAL MODEL | Everything here maps onto work you already do. A PHP-FPM pool is a process pool. A Laravel queue worker is a long-lived process competing for CPU time. A MySQL deadlock is Coffman's four conditions in a database engine. Redis is single-threaded to *avoid* everything in Section 3. Keep making that connection as you read — it is what turns theory into judgement.

Prerequisite vocabulary (30 seconds)

Term	Meaning in one line
Kernel	The privileged core of the OS: owns the CPU, memory, devices, and scheduling.
User space / kernel space	Two privilege levels. Your code runs in user space and must ask the kernel for anything privileged.
System call (syscall)	The controlled doorway from user space into the kernel: <code>read</code> , <code>write</code> , <code>fork</code> , <code>mmap</code> .
Trap / interrupt	A CPU-level event that jumps into the kernel. Traps are synchronous (caused by your instruction), interrupts are asynchronous (caused by hardware).
Context	The CPU register set + memory mappings that define "what is currently running".
Preemption	The kernel forcibly taking the CPU away from a running task.

1. Processes vs Threads

A **process** is a running program plus everything the OS has given it: an address space, open file descriptors, a PID, credentials, and at least one thread of execution. A **thread** is a single flow of execution *inside* a process. The critical distinction is not "threads are lighter" — it is **what is shared and what is isolated**.

1.1 What each one owns

Resource	Per process	Per thread
Address space (code, heap, globals)	Private — isolated	Shared with siblings
Stack	One per thread	Private (own stack + own registers)
Registers / program counter	Per thread	Private
File descriptor table	Private	Shared
Signal handlers	Private	Shared (disposition), delivery can be per-thread
PID / credentials (uid, gid)	Private	Shared (thread has a TID)
Memory mappings, heap allocator state	Private	Shared
Thread-local storage (TLS)	n/a	Private

MENTAL MODEL | Threads share the heap; they do not share the stack. That single sentence explains 90% of concurrency bugs: a local variable is safe, anything reachable from a global, a static, an object field, or a captured reference is not.

1.2 The kernel's view: PCB, task_struct, and TCB

The kernel tracks each schedulable entity in a control block. Classically the textbook calls it the **PCB** (Process Control Block) and **TCB** (Thread Control Block). Linux collapses both into one structure, `struct task_struct`, and a "process" is simply a group of tasks that share a memory descriptor (`struct mm_struct`) and a thread group ID.

A PCB / `task_struct` holds, roughly:

```
identity      : pid, tgid (thread group id = the "PID" userspace sees), ppid, uid/gid
state         : TASK_RUNNING | INTERRUPTIBLE | UNINTERRUPTIBLE | STOPPED | ZOMBIE
scheduling    : policy, priority/nice, vruntime, cpu affinity mask, time accounting
registers     : saved CPU context (thread_struct) - restored on context switch
memory        : *mm_struct -> page tables, VMAs, brk, mmap regions
files         : *files_struct -> the fd table
signals       : *signal_struct, pending set, handler dispositions
namespaces    : pid/net/mnt/uts/ipc/user (this is what makes containers work)
parent/child  : links used by wait(), and the exit_code left behind for the parent
```

In Linux: two tasks with the SAME `mm_struct` = two threads of one process
 two tasks with DIFFERENT `mm_struct` = two processes

Everything in Sections 5-8 is really about manipulating fields in this structure.

1.3 Why the distinction costs you money

- **Creation cost.** A process needs a new page-table hierarchy and a copy of the mapping metadata. A thread reuses them. Thread creation is typically 5–20x cheaper.

- **Context-switch cost.** Switching between threads of the same process does not need a page-table (CR3) swap, so the TLB is not flushed. Switching processes usually is more expensive because of TLB and cache pressure. See Section 8.5.
- **Failure blast radius.** A segfault or an unrecoverable exception kills the whole process, taking every thread with it. Separate processes contain the damage — this is exactly why PHP-FPM and Unicorn use worker *processes*, not threads.
- **Communication cost.** Threads communicate through shared memory: fast, but requires locks. Processes need IPC (pipes, sockets, shared memory segments, message queues), which costs copying and syscalls but needs no locking for the common cases.
- **Safety of the runtime.** Not every library is thread-safe. Much of the classic PHP ecosystem is not, which is why the mainstream PHP deployment model is process-per-request rather than thread-per-request.

1.4 Threading models

Model	Mapping	Characteristics
1:1 (kernel threads)	Each user thread = one kernel task	What Linux/NPTL, Windows and macOS use. True parallelism, kernel-scheduled, but each thread costs kernel memory (~8–16 KB kernel stack) and switches go through the scheduler.
N:1 (green threads)	Many user threads on one kernel thread	Switching is a userspace function call — extremely cheap. But one blocking syscall blocks everyone, and it cannot use multiple cores.
M:N (hybrid)	M user threads over N kernel threads	Go goroutines, Erlang processes, Java virtual threads. Cheap creation plus real parallelism; the runtime scheduler must handle blocking calls carefully.

1.5 Process vs thread vs async: choosing

Workload	Best fit	Reason
CPU-bound, multi-core	Processes or threads	You need real parallelism. Choose processes if the runtime has a global lock (Python GIL) or is not thread-safe.
I/O-bound, high connection count	Async / event loop	Thousands of sockets, each mostly idle. Threads waste memory and switch cost; an event loop (epoll) handles it in one thread. Node.js, Swoole, Nginx.
Mixed, needs isolation	Process pool	A crashed or leaking worker is recycled without touching the others. PHP-FPM, Unicorn, Laravel Horizon workers.
Shared large in-memory state	Threads	A 4 GB model or cache loaded once and read by many workers. Processes would either duplicate it or need shared memory plumbing.

MENTAL MODEL | Your own stack, read through this lens: **Nginx** = few processes, each an event loop (I/O-bound). **PHP-FPM** = pool of isolated processes (crash containment + non-thread-safe runtime). **Laravel queue worker** = one long-lived process per worker, restarted periodically because PHP was never designed to run forever. **MySQL** = one thread per connection over shared buffer pool (needs locks — hence Section 4). **Redis** = one thread for command execution, so it needs no locks at all and every command is atomic by construction.

1.6 IPC: how separate processes talk

Mechanism	Shape	Notes
Pipe / FIFO	Byte stream, unidirectional	Created by <code>pipe()</code> ; the backbone of shell <code> </code> . Anonymous pipes need a parent-child relationship; FIFOs (named pipes) do not.

Mechanism	Shape	Notes
Unix domain socket	Byte stream or datagram, bidirectional	Same API as TCP but local and much faster. This is how Nginx talks to PHP-FPM over <code>/run/php/php8.4-fpm.sock</code> .
Shared memory	Shared pages	<code>shm_open</code> + <code>mmap</code> . Fastest — zero copy — but you are back to needing locks/semaphores.
Message queue	Discrete messages	POSIX mq or SysV; preserves message boundaries and priority.
Signals	One-bit async notification	Not really data transfer. <code>SIGTERM</code> , <code>SIGHUP</code> , <code>SIGUSR1</code> .
Files / memory-mapped files	Persistent bytes	Simple, durable, needs its own locking (flock).
Network / broker	Messages over TCP	Redis, RabbitMQ, Kafka. IPC that survives machine boundaries.

2. Concurrency

Concurrency is about structure; parallelism is about execution. A concurrent program is one that is *decomposed* into independently progressing tasks. A parallel program is one where those tasks literally run at the same instant on different cores. You can have concurrency on a single core (interleaving), and you can have parallelism without meaningful concurrency (SIMD on a vector).

Concurrency (1 core, interleaved)	Parallelism (2 cores, simultaneous)
core0: AAA BBB AAA BBB AAA	core0: AAAAAAAAAAAAAA
	core1: BBBBBBBBBBBBBBBB
-> tasks make progress "together"	-> tasks execute at the same instant
-> hides latency (I/O waits)	-> increases throughput (CPU work)

MENTAL MODEL | Concurrency buys you **latency hiding**. Parallelism buys you **throughput**. If your API endpoint is slow because it makes six sequential HTTP calls, you need concurrency. If it is slow because it hashes a million rows, you need parallelism.

2.1 The three hazards

a) Race condition

Two or more flows access shared mutable state, at least one writes, and the outcome depends on timing. The canonical example is that `counter++` is not one operation — it is three.

`counter++` compiles to roughly:

```
mov  eax, [counter]    ; 1. LOAD
add  eax, 1            ; 2. MODIFY
mov  [counter], eax    ; 3. STORE
```

Two threads, counter starts at 0, both run `counter++` :

```
T1: LOAD  (eax=0)
T2: LOAD  (eax=0)      <-- interleaved here
T1: ADD   (eax=1)
T2: ADD   (eax=1)
T1: STORE (counter=1)
T2: STORE (counter=1)  <-- one increment LOST
```

Expected 2, got 1. This is a "lost update" / read-modify-write race.

b) Time-of-check to time-of-use (TOCTOU)

You check a condition and then act on it, but the world changes in between. This is both a correctness bug and a classic security vulnerability.

```
BROKEN (races between check and use):
if (!file_exists($path)) {           // check
    file_put_contents($path, $x);    // use  <- another process created it meanwhile
}
```

```
BROKEN in the DB layer (the "get-or-create" race):
$u = User::where('email',$e)->first();
if (!$u) User::create([...]);        // two requests both see null -> duplicate rows
```

```
FIXED - make check+act one atomic operation:
- filesystem : fopen($path, 'x')      // O_CREAT|O_EXCL, atomic in the kernel
- database   : UNIQUE INDEX + INSERT ... ON DUPLICATE KEY UPDATE
               or SELECT ... FOR UPDATE inside a transaction
               or Laravel's User::firstOrCreate() backed by a unique index
- redis      : SET key val NX EX 30   // atomic test-and-set = a distributed lock
```

c) Memory visibility and reordering

Even with no interleaving problem, a write made by one thread may not be *visible* to another. Each core has its own store buffer and caches, and both the compiler and the CPU are allowed to reorder instructions as long as single-threaded behaviour is preserved. Without a synchronisation primitive there is no guarantee about when, or in what order, another thread sees your writes.

```
Thread A          Thread B
data = 42;        while (!ready) { }
ready = true;     print(data);    // may print 0 !
```

Why: the CPU/compiler may commit 'ready' before 'data', or B may read a stale cached 'data'. Marking 'ready' volatile is NOT enough in C/C++.

Fix: a release-store on A and an acquire-load on B - which is exactly what unlocking and locking a mutex give you for free. Locks provide both MUTUAL EXCLUSION and MEMORY VISIBILITY (a happens-before edge).

GOTCHA | This is why "I will just use a boolean flag instead of a lock" is usually wrong. The lock is not only preventing interleaving — it is publishing your writes.

2.2 Critical section and the three correctness requirements

A **critical section** is the region of code that touches shared state and must not be executed by two flows at once. Any valid solution must satisfy:

- 1 **Mutual exclusion** — at most one flow inside the critical section.
- 2 **Progress** — if no one is inside and someone wants in, the choice of who enters cannot be postponed indefinitely by threads that are not trying to enter.
- 3 **Bounded waiting** — there is a limit on how many times others can enter before a waiting thread gets its turn (no starvation).

2.3 Concurrency models you will actually meet

Model	Idea	Where you see it
Shared memory + locks	Threads mutate shared state, guarded by mutexes	Java, C++, C#, MySQL internals. Powerful, easy to get wrong.
Message passing / actors	No shared state; send immutable messages	Erlang/Elixir, Akka, Go channels. Removes data races by construction.
Event loop / async	Single thread, non-blocking I/O, callbacks or coroutines multiplexed by epoll/kqueue	Node.js, Nginx, Python asyncio, Swoole, ReactPHP. No data races within the loop, but a blocking call stalls everything.
Fork-join / data parallel	Split data, process independently, merge	MapReduce, OpenMP, array_chunk + parallel jobs.
Software transactional memory	Optimistic: run, detect conflict, retry	Clojure refs, Haskell STM. Same idea as your DB's optimistic locking.

2.4 Amdahl's law: the ceiling on parallelism

If a fraction p of a program can be parallelised and the rest is inherently serial, the maximum speedup with N cores is bounded:

$$\text{Speedup}(N) = 1 / ((1 - p) + p/N) \quad \text{and as } N \rightarrow \text{infinity: } 1 / (1 - p)$$

```
p = 0.50 -> max speedup 2x    (16 cores gives 1.88x)
p = 0.90 -> max speedup 10x   (16 cores gives 6.4x)
p = 0.95 -> max speedup 20x   (16 cores gives 9.1x)
p = 0.99 -> max speedup 100x  (16 cores gives 13.9x)
```

The practical lesson: **the serial fraction is the enemy**, and locks, single database connections, and shared counters are exactly what create it. Adding workers to a queue that all contend on one row-level lock will flatten out fast — and may go backwards once contention overhead exceeds the gain.

2.5 Concurrency in the PHP world specifically

- Classic PHP-FPM gives you **process-level concurrency for free**: N workers, N simultaneous requests, no shared memory, no data races inside a request.
- The shared state you actually race on is **external**: MySQL rows, Redis keys, files, S3 objects. So your concurrency control lives there — DB transactions, unique indexes, `SELECT ... FOR UPDATE`, and `Cache::lock()` (Redis `SET NX`).
- Laravel Horizon / `queue:work` scales by process count; use `WithoutOverlapping` and `ShouldBeUnique` to serialise jobs that touch the same entity.
- Real in-process concurrency exists via **Swoole/OpenSwoole** (coroutines), **parallel** (ext-parallel, true threads on ZTS builds), **Fibers** (PHP 8.1, cooperative, single-threaded), and **Octane** (long-lived workers — which suddenly makes static/global state a real bug source, because it now survives between requests).

GOTCHA | Octane changes the rules. Under FPM a leaked static is cleaned up when the request ends. Under Octane the worker lives on, so a static cache, a container singleton holding request state, or a global array becomes both a memory leak and a cross-request data leak.

3. Locks and Mutex

A lock is a mechanism that enforces mutual exclusion over a critical section. Every lock must be built on a **hardware atomic instruction**, because you cannot build atomicity out of non-atomic parts in software alone (Peterson's algorithm can, but it needs memory barriers and does not scale).

3.1 The hardware foundation: atomic instructions

Test-And-Set (TAS)

atomically:

```
old = *ptr
*ptr = 1
return old
```

Compare-And-Swap (CAS)

atomically:

```
if (*ptr == expected) { *ptr = new; return true; }
else { expected = *ptr; return false; }
```

x86: XCHG, LOCK CMPXCHG

ARM: LDREX/STREX, LDAXR/STLXR (load-linked/store-conditional)

CAS is the more powerful primitive: it is the basis of every lock-free data structure, of `std::atomic`, of Java's `AtomicInteger`, and of the "optimistic locking" pattern (WHERE version = ? in your UPDATE).

3.2 Mutex vs the alternatives

Primitive	Semantics	Use when
Mutex	Binary, has an <i>owner</i> . Only the locking thread may unlock. Waiters sleep (blocked, off the run queue).	General mutual exclusion around a critical section held for more than a few hundred nanoseconds.
Spinlock	Binary, waiters <i>busy-wait</i> in a tight loop, burning CPU. No context switch.	Extremely short critical sections, multi-core, and where sleeping is illegal (inside interrupt handlers in the kernel). Never on a single core.
Binary semaphore	Count of 1, but <i>no ownership</i> — any thread may signal it.	Signalling between threads ("the data is ready"), not protecting data.
Counting semaphore	Holds N permits. wait/P takes one, signal/V returns one.	Resource pools and rate limiting: N database connections, N concurrent uploads.
Read-write lock	Many concurrent readers OR one exclusive writer.	Read-heavy shared structures (a config map, a route table). Beware writer starvation.
Condition variable	Wait for a <i>predicate</i> to become true; always paired with a mutex.	Producer/consumer, bounded queues, "wait until the queue is non-empty".
Atomic variable	Lock-free single-word read-modify-write.	Counters, flags, simple state machines. No blocking, no deadlock.
RCU	Read-Copy-Update: readers never block; writers publish a new copy and defer freeing the old one.	Extremely read-dominated kernel structures. Used heavily inside Linux.

MENTAL MODEL | **Mutex protects data. Semaphore counts permits. Condition variable waits for a state change.** Interviewers love the mutex-vs-semaphore question; the discriminator to say out loud is **ownership** — a mutex has an owner and therefore supports priority inheritance and error checking; a semaphore does not.

3.3 Using a mutex correctly

```

/* C / pthreads */
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
long counter = 0;

void *worker(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&m);
        counter++;
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

/* Condition variable: the ONLY correct wait pattern */
pthread_mutex_lock(&m);
while (queue_is_empty(q)) /* WHILE, never IF - guards against spurious */
    pthread_cond_wait(&cv, &m); /* atomically unlocks m, sleeps, relocks on wake */
item = dequeue(q);
pthread_mutex_unlock(&m);

/* producer side */
pthread_mutex_lock(&m);
enqueue(q, item);
pthread_cond_signal(&cv); /* or _broadcast() if multiple waiters may proceed */
pthread_mutex_unlock(&m);

```

The while loop is mandatory: wakeups can be spurious, and another thread may have consumed the item between the signal and your reacquiring the lock.

3.4 The rules that keep you out of trouble

- **Hold locks for as short a time as possible.** Never do I/O, a network call, or a DB query while holding an in-memory lock.
- **Never call unknown code while holding a lock** (callbacks, event listeners, virtual methods). You do not know what *they* will lock.
- **Always acquire multiple locks in a fixed global order.** This alone prevents most deadlocks (Section 4).
- **Use RAIL / scope guards** so the lock is released even on an exception: `std::lock_guard`, Go's `defer mu.Unlock()`, PHP's `try/finally`.
- **Prefer immutability and message passing** to locking where you can. The fastest lock is the one you did not need.
- **Beware lock granularity.** One coarse lock is simple but serialises everything; many fine locks scale but multiply deadlock risk. Start coarse, measure, then split.

3.5 Failure modes to name in an interview

Failure	What happens
Deadlock	Two or more flows each wait for a lock the other holds. Nobody proceeds. See Section 4.
Livelock	Threads keep changing state in response to each other but make no progress — two people stepping aside in a corridor, forever.
Starvation	A thread is perpetually denied the resource: low priority, or an RWLock where a steady stream of readers never lets the writer in.
Priority inversion	A low-priority thread holds a lock a high-priority thread needs, while a medium-priority thread preempts the low one. Fixed by priority inheritance (this bug famously stalled Mars Pathfinder in 1997).
Convoy effect	One slow lock holder makes every other thread queue up behind it, destroying throughput even after it releases.
Lock contention / thrashing	So much time spent acquiring and switching that adding threads makes it slower.

Failure	What happens
ABA problem	In lock-free CAS code a value changes $A \rightarrow B \rightarrow A$, so CAS succeeds although the world changed. Fixed with tagged pointers or hazard pointers.

3.6 Locking in your day job

```
# --- Redis / Laravel: distributed lock (mutual exclusion across MACHINES) ---
$lock = Cache::lock('settle-invoice:'.$id, 10); // 10s TTL = automatic deadlock recovery
if ($lock->get()) {
    try { $this->settle($id); } finally { $lock->release(); }
}
// or block up to 5s waiting: Cache::lock($k, 10)->block(5, fn() => $this->settle($id));

# --- MySQL: pessimistic row lock (exclusive, held to end of transaction) ---
DB::transaction(function () use ($id) {
    $seat = Seat::where('id',$id)->lockForUpdate()->first(); // SELECT ... FOR UPDATE
    abort_if($seat->status !== 'free', 409);
    $seat->update(['status' => 'booked']);
});

# --- MySQL: optimistic lock (CAS, no lock held; good for low contention) ---
$n = DB::update('UPDATE seats SET status=?, version=version+1
    WHERE id=? AND version=?', ['booked', $id, $expectedVersion]);
if ($n === 0) throw new ConflictException('someone else changed it'); // retry

# --- Laravel queue: serialise jobs touching the same entity ---
public function middleware() { return [new WithoutOverlapping($this->tripId)]; }
```

Seat booking on a ticketing platform is the textbook critical section: a check (is it free?) followed by an act (book it) that must be atomic.

GOTCHA | A distributed lock with a TTL is not the same as a mutex. If your work outlives the TTL, a second worker acquires the lock while you are still running. Either keep the critical section far shorter than the TTL, renew the lease (fencing tokens), or make the operation idempotent so a double execution is harmless.

4. Deadlock

A **deadlock** is a state in which a set of processes are each blocked waiting for an event that only another process in the same set can cause. No one in the set will ever proceed without outside intervention.

4.1 The four Coffman conditions (all four must hold)

#	Condition	Meaning	Break it by
1	Mutual exclusion	At least one resource is held in a non-shareable mode.	Make the resource shareable / read-only / lock-free. Often impossible.
2	Hold and wait	A process holding a resource requests another.	Acquire <i>all</i> resources at once (all-or-nothing), or release everything before requesting more.
3	No preemption	A resource cannot be forcibly taken from its holder.	Allow rollback: if a request fails, release what you hold and retry. This is what databases do.
4	Circular wait	A cycle exists in the wait-for graph: $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_1$.	Impose a global lock ordering and always acquire in that order. Simplest and most practical fix.

MENTAL MODEL | Remember the four as **M-H-N-C**: Mutual exclusion, Hold-and-wait, No preemption, Circular wait. Deadlock needs all four; removing *any one* makes it impossible. In practice you almost always attack #4 with a lock ordering, or #3 with timeouts and retries.

4.2 The canonical example

```

Thread A                      Thread B
lock(mutex1)                  lock(mutex2)
... (context switch here) ...
lock(mutex2) <-- blocked      lock(mutex1) <-- blocked

Wait-for graph:      A ---waits for---> B
                    ^                   |
                    +---waits for-----+    = cycle = deadlock

```

FIX (lock ordering): every thread locks the LOWER-addressed / lower-id mutex first.
 Thread A: lock(mutex1); lock(mutex2)
 Thread B: lock(mutex1); lock(mutex2) <-- now B waits, but no cycle can form

The same bug in application terms:
 transfer(from=101, to=205) locks row 101 then 205
 transfer(from=205, to=101) locks row 205 then 101 -> deadlock
 Fix: always lock accounts in ascending id order, regardless of direction.

4.3 The four strategies

Strategy	How	Cost / reality
Prevention	Structurally negate one Coffman condition (usually circular wait, via lock ordering).	Cheap, static, no runtime overhead. The right answer for application code.
Avoidance	Grant a request only if the system stays in a <i>safe state</i> — Banker's algorithm.	Requires knowing every process's maximum future claim up front. Almost never practical outside a textbook or a fixed embedded system.
Detection & recovery	Let deadlocks happen; periodically look for a cycle in the wait-for graph, then kill/roll back a victim.	What MySQL InnoDB does. Recovery = pick the transaction that has done the least work and roll it back.

Strategy	How	Cost / reality
Ostrich algorithm	Ignore it. Reboot or restart when it happens.	Genuinely the choice made by most general-purpose OS kernels for user-level deadlocks — the cost of prevention exceeds the cost of the rare hang.

Banker's algorithm in one pass

Given: Allocation[i][j] resources of type j currently held by process i
 Max[i][j] maximum process i will ever need
 Available[j] free instances of resource j
 Need = Max - Allocation

SAFETY CHECK (is there a sequence in which everyone can finish?):

```
work = Available; finish[] = false
repeat:
  find i with finish[i]==false and Need[i] <= work      (it can complete)
  if found: work += Allocation[i]; finish[i] = true      (it finishes, frees its resources)
  else: break
SAFE if all finish[i] == true, else UNSAFE.
```

A request is granted only if, after pretending to grant it, the state is still safe.

Note: unsafe != deadlocked. Unsafe means deadlock has become POSSIBLE.

4.4 Database deadlocks — the ones you will actually hit

InnoDB detects deadlocks automatically and rolls back one transaction with **ERROR 1213 (40001): Deadlock found when trying to get lock. Your job is (a) to reduce their frequency and (b) to retry safely when one happens.**

Common causes

- Two transactions updating the same rows in opposite order
- Range locks from a non-indexed WHERE (gap/next-key locks over many rows)
- Long transactions holding locks while doing slow work
- Mixing lockForUpdate with plain updates
- Hot-row contention (a counters table)

Practical fixes

- Touch rows in a consistent (e.g. ascending PK) order in every code path
- Index the columns you filter and update on; an unindexed UPDATE can lock the whole table
- Keep transactions short; do HTTP calls, mail, and file I/O OUTSIDE the transaction
- Standardise on one locking strategy per table
- Move counters to Redis, or shard the row

Diagnose:

```
SHOW ENGINE INNODB STATUS\G
-> "LATEST DETECTED DEADLOCK"
SELECT * FROM performance_schema.data_locks;
```

Retry (deadlock is a TRANSIENT error - safe to retry):

```
DB::transaction(function () { ... }, 3); // 3 attempts
// exponential backoff + jitter for external systems
```

On a ticketing system, the classic pattern is two bookings locking seat rows in different orders. Sorting seat IDs before locking removes the cycle entirely.

GOTCHA | Deadlock (cyclic wait, resolved by rollback) is not the same as **lock wait timeout** (ERROR 1205 — one transaction simply held a lock too long). Deadlock is a design problem in ordering; lock wait timeout is usually a long-transaction problem.

4.5 Detecting deadlocks in your own code

```
# Which threads are stuck, and where?
gdb -p <PID> -batch -ex "thread apply all bt"      # C/C++ - look for pthread_mutex_lock frames
jstack <PID>                                       # Java - prints "Found one Java-level deadlock"
cat /proc/<PID>/task/*/stack                       # kernel-side stack of every thread
ps -eo pid,stat,wchan:32,comm | grep ' D '        # D state = uninterruptible sleep (often I/O)
```

Symptoms: CPU near 0% but the process is unresponsive, thread count flat, no progress in logs.
 # (100% CPU with no progress is LIVELOCK or a spin, not a deadlock.)

5. Scheduling

The scheduler decides **which runnable task gets a CPU next, and for how long**. It is the OS component that most directly shapes the latency your users feel.

5.1 Vocabulary and metrics

Metric	Definition	Who cares
Arrival time	When the task became runnable.	—
Burst time	CPU time the task needs before it blocks or finishes.	—
Completion time	When it finished.	—
Turnaround	Completion – Arrival (total time in the system).	Batch jobs
Waiting time	Turnaround – Burst (time spent runnable but not running).	Fairness
Response time	First time it got the CPU – Arrival.	Interactive apps
Throughput	Tasks completed per unit time.	Servers, batch
CPU utilisation	Fraction of time the CPU is doing useful work.	Capacity planning

These goals conflict. Optimising average turnaround favours short jobs; optimising response time favours frequent preemption, which costs context switches and throughput. Every real scheduler is a compromise.

5.2 Classic algorithms

Algorithm	Rule	Pros / Cons
FCFS (FIFO)	Run in arrival order, non-preemptive.	Trivial, no starvation. Suffers the <i>convoy effect</i> : one long job delays everyone. Terrible response time.
SJF	Shortest job first, non-preemptive.	Provably optimal average waiting time. But burst time is unknown in advance, and long jobs can starve.
SRTF	Shortest remaining time first — preemptive SJF.	Even better average waiting time; more switches; worse starvation for long jobs.
Round Robin	Each task gets a fixed time quantum, then goes to the back of the queue.	Fair, great response time, no starvation. Quantum too small → switch overhead dominates; too large → degenerates to FCFS. Typical: 10–100 ms.
Priority	Highest priority runs first (preemptive or not).	Expresses importance. Causes starvation — fixed by ageing (raise priority the longer you wait).
MLFQ	Multiple queues at different priorities; a task that uses its full quantum drops a level, a task that yields early stays high. Periodic boost.	Approximates SJF <i>without knowing burst times</i> , by learning behaviour. This is the design behind classic Unix and Windows schedulers.
Fair share / CFS	Track how much CPU each task has received; always run the one that has received least (weighted by priority).	Proportional fairness with no fixed quantum. Linux 2.6.23–6.5.
Lottery / stride	Give tickets in proportion to share; draw at random (lottery) or deterministically (stride).	Simple, elegant proportional sharing.

5.3 Worked example — compute these by hand

Process	Arrival	Burst
P1	0	7
P2	2	4
P3	4	1
P4	5	4

FCFS: | P1 0-7 | P2 7-11 | P3 11-12 | P4 12-16 |
 turnaround = 7-0, 11-2, 12-4, 16-5 = 7, 9, 8, 11 -> avg 8.75
 waiting = turnaround - burst = 0, 5, 7, 7 -> avg 4.75

SRTF (preemptive SJF):
 | P1 0-2 | P2 2-4 | P3 4-5 | P2 5-7 | P4 7-11 | P1 11-16 |
 completion: P1=16, P2=7, P3=5, P4=11
 turnaround = 16, 5, 1, 6 -> avg 7.00
 waiting = 9, 1, 0, 2 -> avg 3.00 <-- optimal

Round Robin, quantum = 2:
 | P1 0-2 | P2 2-4 | P1 4-6 | P3 6-7 | P2 7-9 | P4 9-11 | P1 11-13 | P4 13-15 | P1 15-16 |
 completion: P1=16, P2=9, P3=7, P4=15
 turnaround = 16, 7, 3, 10 -> avg 9.00
 waiting = 9, 3, 2, 6 -> avg 5.00
 ...but RESPONSE time is far better: P1=0, P2=0, P3=2, P4=4 -> avg 1.5

The takeaway: SRTF wins on turnaround, Round Robin wins on responsiveness. Which one you want depends on whether a human is waiting.

5.4 Linux in practice: CFS, EEVDF, and the scheduling classes

Linux does not use one algorithm; it uses **scheduling classes** consulted in strict priority order. A runnable real-time task always beats every normal task.

Class / policy	Behaviour
SCHED_DEADLINE	Earliest-Deadline-First with a runtime budget and period. Highest priority of all. For hard real-time work.
SCHED_FIFO / SCHED_RR	POSIX real-time, priorities 1–99. FIFO runs until it blocks or yields; RR adds a time slice. A runaway FIFO task can lock up a core.
SCHED_OTHER (a.k.a. NORMAL)	The default. CFS until Linux 6.5, EEVDF (Earliest Eligible Virtual Deadline First) from 6.6 onwards. Uses <i>nice</i> values –20 (greediest) to +19 (nicest); each nice level is roughly a 1.25x weight change.
SCHED_BATCH	Like NORMAL but assumes the task is CPU-bound and non-interactive; the scheduler stops giving it wakeup bonuses.
SCHED_IDLE	Runs only when nothing else wants the CPU.

How CFS thinks (the key idea to be able to explain)

Each task has a "vruntime" = weighted virtual runtime it has consumed:

```
vruntime += actual_runtime * (NICE_0_WEIGHT / task_weight)
```

- a high-priority task (low nice, big weight) accumulates vruntime SLOWLY
-> so it stays at the front and gets picked more often
- all runnable tasks sit in a red-black tree keyed by vruntime; $O(\log n)$
- the scheduler always picks the LEFTMOST node = smallest vruntime
- a newly woken task gets $vruntime = \max(\text{its own}, \text{min_vruntime} - \text{latency})$
so a sleeping (interactive) task returns with a small vruntime and preempts
-> this is why your shell stays responsive under a kernel compile

EEVDF (6.6+) adds an explicit "lag" and a virtual DEADLINE per task, which gives much better latency guarantees for tasks that ask for short slices.

Tuning it from the command line

```

nice -n 10 php artisan queue:work      # start a low-priority worker (nice 10)
renice -n 5 -p 4321                    # change priority of a running process
renice -n -5 -p 4321                   # needs root; negative nice = MORE cpu

chrt -f -p 50 4321                     # move pid to SCHED_FIFO priority 50 (careful!)
chrt -p 4321                           # show current policy and priority

taskset -c 2,3 ./cpu_bound              # pin to cores 2 and 3 (cache affinity, NUMA)
taskset -pc 4321                       # show affinity of a running process

# cgroups v2 = the real production tool (this is how containers limit CPU)
cat /sys/fs/cgroup/<slice>/cpu.max      # e.g. "200000 100000" = 2 CPUs worth
cat /sys/fs/cgroup/<slice>/cpu.stat     # nr_throttled, throttled_usec <-- check this!

```

GOTCHA | If a containerised PHP app shows random latency spikes at low CPU usage, look at `cpu.stat` for `nr_throttled`. CFS bandwidth control hands out quota in 100 ms periods; a bursty worker can exhaust its quota in 20 ms and then sit frozen for 80 ms. The fix is usually a larger quota or fewer threads, not more replicas.

5.5 I/O-bound vs CPU-bound, and why schedulers favour the former

- An **I/O-bound** task uses a short CPU burst then blocks. Giving it the CPU immediately costs almost nothing and keeps devices busy, so every good scheduler prioritises it.
- A **CPU-bound** task consumes its whole quantum. It cares about throughput, not latency.
- MLFQ and CFS both detect this from *behaviour*, not from a declaration: a task that sleeps a lot is inferred to be interactive.
- Practical consequence: run your CPU-heavy report generation at `nice 10` or in a separate cgroup so it cannot degrade request-serving workers.

6. Memory Management

Memory management is the OS's job of deciding which parts of which processes live in physical RAM, where, and for how long — while giving every process the illusion of a large, private, contiguous address space.

6.1 The memory hierarchy (know the numbers)

Level	Typical latency	Typical size	Notes
CPU register	< 1 ns	a few hundred bytes	Compiler-managed.
L1 cache	~1 ns (4 cycles)	32–64 KB/core	Split I-cache and D-cache.
L2 cache	~4 ns	256 KB–2 MB/core	
L3 cache	~12–40 ns	8–64 MB shared	Shared across cores.
Main memory (DRAM)	~80–100 ns	8 GB–1 TB	~100x slower than L1.
NVMe SSD	~50–150 μ s	0.5–8 TB	~1000x slower than DRAM.
Spinning disk	~5–10 ms	TBs	~100,000x slower than DRAM.

The whole design of caches, TLBs, page replacement and buffer pools exists because of those ratios. **Locality** — temporal (you will touch it again soon) and spatial (you will touch its neighbour soon) — is what makes the hierarchy work.

6.2 The address space of a Linux process

```

high addresses
0x7fff_ffff_ffff +-----+
                  | kernel space (not mapped | <- ring 0 only; mapped but inaccessible
                  | into user page tables   | (KPTI splits this since Meltdown)
                  | post-KPTI)              |
                  +-----+
                  | [stack] grows DOWN     | <- locals, return addresses, saved registers
                  | |                       | one per thread; default 8 MB (ulimit -s)
                  | v                       |
                  | memory-mapped region    | <- unmapped gap: touching it = SIGSEGV
                  | ^                       | <- shared libs (libc.so), mmap(), file maps
                  | |                       |
                  | [heap] grows UP         | <- malloc/new; extended by brk() or mmap()
                  +-----+
                  | .bss (zero-initialised |
                  | .data (initialised    |
                  | .rodata (constants,   |
                  | .text (machine code,  |
                  |                       |
0x0000_0000_0000 +-----+
low addresses     page 0 deliberately unmapped -> dereferencing NULL faults immediately

```

Inspect any live process with `cat /proc/<pid>/maps` or `pmap -x <pid>`.

MENTAL MODEL | Stack vs heap is not a language detail, it is this diagram. Stack: automatic lifetime, LIFO, fast (bump a pointer), small and fixed, per-thread. Heap: manual or GC lifetime, arbitrary order, slower (needs an allocator), large, shared across threads — which is exactly why heap data needs locks and stack data does not.

6.3 Allocation strategies (historical, but still the vocabulary)

Contiguous allocation and its placement policies

- **First fit** — take the first hole big enough. Fast, decent.

- **Best fit** — take the smallest adequate hole. Wastes less, but leaves many unusably tiny slivers and is slower to search.
- **Worst fit** — take the largest hole. Generally the worst of the three.
- **Next fit** — first fit, but resume searching where you stopped last time.

Fragmentation

Type	Cause	Remedy
External	Free memory exists but is split into non-contiguous holes, so a large request fails despite sufficient total free space.	Compaction (expensive), paging (eliminates the requirement for contiguity), or the buddy allocator.
Internal	Allocation is rounded up to a fixed block/page size; the unused remainder inside the block is wasted.	Smaller pages or a slab allocator with size classes. Never fully eliminated.

Kernel allocators worth naming

- **Buddy allocator** — free memory is kept in power-of-two blocks. A request is satisfied by splitting a block in half repeatedly ("buddies"); on free, buddies are merged back. Fast coalescing, low external fragmentation, but up to 2x internal fragmentation in the worst case. See `/proc/buddyinfo`.
- **Slab / SLUB allocator** — sits on top of the buddy allocator and caches pre-initialised objects of one fixed type (inodes, task_structs, dentries), avoiding both fragmentation and re-initialisation cost. See `slabtop` and `/proc/slabinfo`.
- **User-space malloc** (glibc ptmalloc, jemalloc, tcmalloc, mimalloc) — requests big chunks from the kernel via `brk/mmap` and then subdivides them itself, using per-thread arenas and size-class bins to reduce lock contention. This is why `free()` usually does *not* return memory to the OS.

GOTCHA | "My PHP worker's RSS never goes down after a big job." That is normally the allocator, not a leak: freed memory is retained in the arena for reuse. It is also the reason for `--max-jobs / --max-time` on queue workers and `pm.max_requests` on PHP-FPM — recycling the process is the cheapest way to actually give memory back.

6.4 Segmentation vs paging

	Segmentation	Paging
Unit	Variable-length, logically meaningful (code, stack, heap)	Fixed-size frames/pages (typically 4 KB)
Address	segment selector + offset	page number + offset
Fragmentation	External	Internal (last page of each region)
Sharing / protection	Natural — permissions per logical segment	Per page, via page-table bits
Status today	Largely vestigial on x86-64 (flat model)	Universal

Modern x86-64 keeps segmentation registers but uses a flat model where the segment base is zero, so **paging does all the real work**. That is Section 7.

6.5 Measuring memory correctly

Metric	Means	Trap
VSZ / VIRT	Total virtual address space reserved.	Wildly overstates real usage — includes untouched mappings and shared libraries. Mostly meaningless for capacity planning.
RSS / RES	Resident set size: physical pages currently mapped in.	Double-counts shared pages across processes. Summing RSS over 40 FPM workers vastly overstates real usage.

Metric	Means	Trap
PSS	Proportional set size: private pages + shared pages divided by the number of sharers.	The correct metric when summing across a process pool. smem or /proc/<pid>/smaps_rollup.
USS	Unique set size: memory that would be freed if this process died.	The honest "cost" of one extra worker.
available (in free -m)	Estimate of memory obtainable without swapping.	Use this, not "free". Page cache is reclaimable, so low "free" is healthy, not a problem.

```

free -h                                # look at "available", not "free"
ps -eo pid,rss,vsz,comm --sort=-rss | head -15
pmap -x <pid>                          # per-mapping breakdown
cat /proc/<pid>/status | grep -E 'VmRSS|VmHWM|VmSwap|Threads' # VmHWM = peak RSS
cat /proc/<pid>/smaps_rollup            # includes Pss, Private_Dirty, Swap
smem -k -P php-fpm                    # PSS/USS per process (apt install smem)
slabtop -o | head                      # kernel slab usage
cat /proc/meminfo                      # the full picture: Cached, Dirty, Slab, SwapFree...

```

6.6 The OOM killer

When Linux cannot reclaim enough memory, the out-of-memory killer selects a victim and kills it. Selection is driven by an oom_score derived mainly from memory footprint, adjustable per process.

```

dmesg -T | grep -i -E 'oom|killed process' # did the OOM killer fire?
journalctl -k | grep -i 'out of memory'
cat /proc/<pid>/oom_score /proc/<pid>/oom_score_adj # -1000 = never kill, +1000 = kill first

```

Overcommit: Linux lets you malloc more than exists, betting you will not touch it all.
 /proc/sys/vm/overcommit_memory 0 = heuristic (default), 1 = always allow, 2 = strict
 Consequence: malloc() succeeding does not mean the memory exists. The failure arrives later, as a SIGKILL from the OOM killer when you first WRITE to the page.

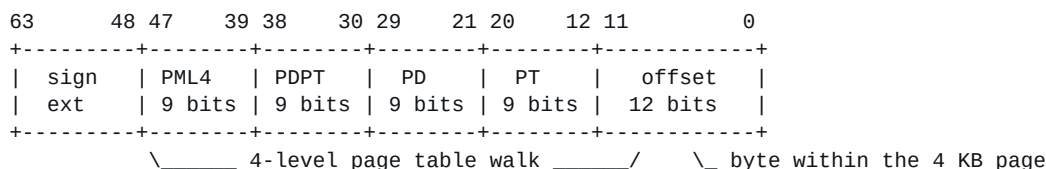
In containers: memory.max in cgroup v2. Exceeding it triggers a cgroup-local OOM kill, which is why a pod dies with exit code 137 (128 + SIGKILL 9) while the host has free RAM.

7. Virtual Memory and Paging

Virtual memory gives every process its own private virtual address space, translated to physical memory by hardware on every single access. It delivers four things at once: **isolation** (process A cannot address process B's memory), **the illusion of more RAM than exists** (via swap and demand paging), **relocation** (a program can be loaded anywhere), and **sharing** (one physical copy of libc mapped into hundreds of processes).

7.1 Address translation

A 48-bit virtual address on x86-64 with 4 KB pages:



offset 12 bits = 4096 bytes per page. 9 bits = 512 entries per table level.
 CR3 register holds the physical address of the current process's PML4.
 Switching processes = loading a new CR3 = a different translation universe.
 (5-level paging exists for 57-bit address spaces on newer CPUs.)

Why multi-level? A flat table for 48 bits at 8 bytes/entry would need 512 GB PER PROCESS.
 A hierarchy only allocates the sub-tables that are actually used - sparse address spaces cost almost nothing.

Page table entry (PTE) bits worth knowing

Bit	Name	Meaning
P	Present	If 0, the page is not in RAM → access raises a page fault .
R/W	Writable	If 0, a write faults. Used for read-only data <i>and</i> copy-on-write.
U/S	User/Supervisor	If 0, only kernel mode may access it.
A	Accessed	Set by hardware on any access. Used by page-replacement (clock) algorithms.
D	Dirty	Set by hardware on write. A clean page can be evicted without writing it back.
NX	No-Execute	Prevents executing data. The hardware basis of DEP / W ^X .

7.2 The TLB — why translation is not ruinously slow

A four-level walk means four extra memory accesses per memory access. The **Translation Lookaside Buffer** is a small, very fast, fully-associative cache of recent virtual→physical translations sitting inside the MMU.

Effective access time with a TLB:

$$EAT = h * (t_{tlb} + t_{mem}) + (1 - h) * (t_{tlb} + (L+1) * t_{mem})$$

h = TLB hit ratio, L = page table levels, t_{mem} = memory access time

Example: $h = 0.98$, $t_{tlb} = 1$ ns, $t_{mem} = 100$ ns, $L = 4$
 hit : $1 + 100 = 101$ ns (98% of the time)
 miss : $1 + 5 * 100 = 501$ ns (2% of the time)
 EAT : $0.98 * 101 + 0.02 * 501 = 109$ ns → only 9% overhead thanks to a 98% hit rate

TLB is tiny (64-1500 entries). With 4 KB pages, 1024 entries covers just 4 MB of working set - which is why huge pages matter for big-heap workloads.

Context switch between PROCESSES historically flushed the TLB. Modern CPUs tag entries with an address-space id (PCID on x86, ASID on ARM) so the flush is largely avoided.

7.3 Demand paging and the page fault

Pages are not loaded until touched. The first access to an unloaded page traps into the kernel as a **page fault**. Not all faults are equal — this distinction is a high-value interview answer:

Fault type	What happened	Cost
Minor (soft)	The page is in physical memory already — it is in the page cache, or a copy-on-write, or a zero-fill for a fresh anonymous page. Only the mapping had to be created.	microseconds
Major (hard)	The page must be read from disk or swap. The process blocks on I/O.	milliseconds — 1000x worse
Invalid	The address is not in any VMA, or the access violates permissions → SIGSEGV.	process dies

Page fault handling, step by step:

1. CPU raises a fault; hardware saves the faulting address (CR2 on x86) and switches to kernel.
2. Kernel checks the VMA list: is this address legal for this process? If not -> SIGSEGV.
3. If legal but not present: find a free physical frame (reclaim/evict one if necessary).
4. Issue I/O to read the page from the file or swap. THE PROCESS BLOCKS; another task runs.
5. On completion update the PTE (present=1, frame no., perms) and flush the stale TLB entry.
6. Return to user space and RE-EXECUTE the faulting instruction, which now succeeds.

Watch them live:

```
ps -eo pid,min_flt,maj_flt,comm --sort=-maj_flt | head
/usr/bin/time -v ./prog      # "Major (requiring I/O) page faults:"
vmstat 1                    # si/so columns = swap in/out pages per second
sar -B 1                     # ppgin/s, majflt/s
```

7.4 Page replacement algorithms

Algorithm	Rule	Verdict
OPT / MIN	Evict the page that will be used farthest in the future.	Provably optimal but unimplementable — requires knowing the future. Used as a benchmark baseline.
FIFO	Evict the oldest-loaded page.	Simple; ignores usage. Suffers Belady's anomaly : more frames can mean <i>more</i> faults.
LRU	Evict the least recently used page.	Excellent in practice (locality), but exact LRU needs a timestamp or list update on every access — too expensive in hardware.
Clock / Second chance	Circular scan; if the reference bit is 1, clear it and move on; evict the first page with a 0 bit.	A cheap LRU approximation using only the hardware Accessed bit. This is what real kernels do.
LFU / MFU	Evict least (or most) frequently used.	LFU is biased by history — a page popular long ago never leaves. Needs ageing.
Linux: two-list LRU + shrinkers	Active and inactive lists, promoting on second reference, plus multi-generational LRU (MGLRU) in recent kernels.	Resists the classic problem where one big sequential file read evicts your whole working set.

Belady's anomaly — be able to show it

Reference string: 1 2 3 4 1 2 5 1 2 3 4 5 FIFO replacement

with 3 frames -> 9 page faults

with 4 frames -> 10 page faults <-- MORE memory, MORE faults

Stack algorithms (LRU, OPT) provably cannot exhibit this, because the set of pages resident with N frames is always a subset of the set resident with N+1 frames.

7.5 Thrashing and the working set

Thrashing is the state where processes spend more time paging than executing. It happens when the sum of the active **working sets** exceeds physical memory: every process steals frames from the others, causing more faults, causing more stealing. CPU utilisation collapses while disk I/O saturates.

- The **working set** $W(t, \Delta)$ is the set of pages referenced in the last Δ references. If Σ working-set sizes $>$ available frames, thrashing is guaranteed.
- **Symptoms:** low CPU, very high `si/so` in `vmstat`, high `iowait`, huge major-fault counts, and a machine that responds to nothing — often worse than an outright crash.
- **Fixes:** reduce the degree of multiprogramming (fewer workers!), add RAM, use working-set-aware admission control, or turn swap off so the OOM killer resolves it quickly and loudly instead of degrading for hours.
- `vm.swappiness` (0–200, default 60) tunes the preference for evicting anonymous memory versus page cache. Database servers commonly set 1–10.

GOTCHA | If a PHP-FPM box goes unresponsive under load with `pm.max_children` set too high, that is thrashing, not slow code. Each worker's working set is real; 200 workers $\times$ 80 MB does not fit in 8 GB. Sizing rule of thumb: `max_children` $\approx$ (RAM available to PHP) / (average worker RSS), measured, not guessed.

7.6 The features built on top of paging

Feature	How paging enables it
Copy-on-write (CoW)	After <code>fork()</code> , parent and child share every physical page, marked read-only. The first write faults, the kernel copies that one page and makes it writable. Makes <code>fork</code> nearly free and is why <code>fork+exec</code> is not wasteful.
Shared libraries	One physical copy of <code>libc</code> 's <code>.text</code> mapped read-only into every process's page tables.
mmap / memory-mapped I/O	Map a file into the address space and access it as an array; the kernel pages it in on demand. Zero-copy reads, and how databases and <code>sendfile</code> avoid double buffering.
Page cache	Free RAM caches file contents. This is why "free" memory looks low on a healthy Linux box and why the second <code>grep</code> of a file is instant.
Swap	Anonymous pages evicted to disk. A little swap is healthy (it lets the kernel evict genuinely cold pages); heavy swapping is thrashing.
Guard pages	An unmapped page below each thread stack turns stack overflow into a clean <code>SIGSEGV</code> instead of silent corruption.
ASLR	Randomising the base of stack, heap and <code>mmap</code> regions at each <code>exec</code> is trivial once addresses are virtual.
Huge pages	2 MB or 1 GB pages cut page-table depth and TLB pressure dramatically for large heaps. Transparent Huge Pages (THP) does it automatically — but many databases recommend disabling THP because its defragmentation stalls cause latency spikes.

```
# See copy-on-write and demand paging with your own eyes
cat /proc/<pid>/smaps | grep -E '^(Rss|Pss|Shared_Clean|Private_Dirty)' | head
grep -i huge /proc/meminfo
cat /sys/kernel/mm/transparent_hugepage/enabled          # [always] madvise never
sysctl vm.swappiness vm.overcommit_memory vm.dirty_ratio
```

8. Linux Process Basics and Context Switching

8.1 Process lifecycle: fork, exec, wait, exit

Unix separates **creating a process** from **running a program**. `fork()` duplicates the calling process; `execve()` replaces the current program image with a new one. That separation is what makes shell redirection possible: the child can adjust its file descriptors *between* the fork and the exec.

```
pid_t pid = fork();          /* returns TWICE: 0 in child, child's PID in parent, -1 on error */

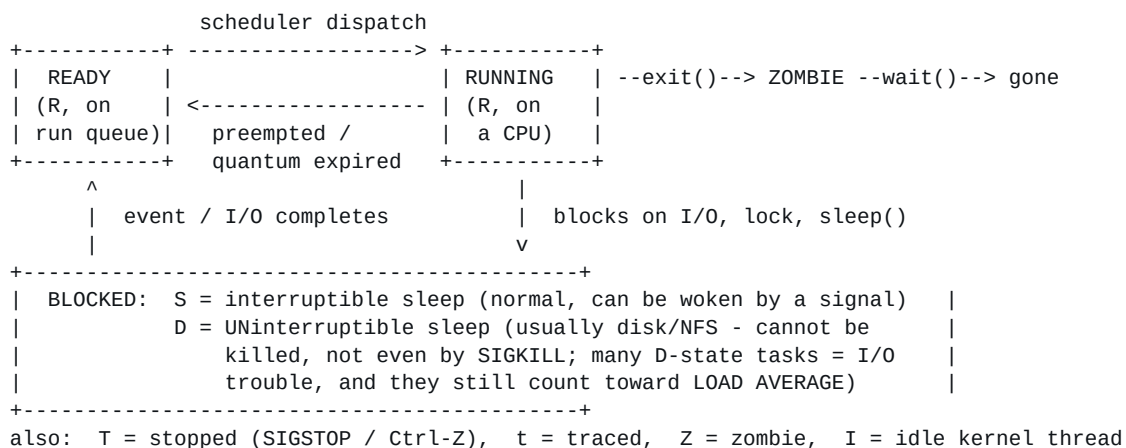
if (pid == 0) {
    /* CHILD: own PID, copy-on-write copy of memory, inherits fds, cwd, env, signal dispositions */
    dup2(fd, STDOUT_FILENO);          /* this is how "cmd > file" works */
    execve("/usr/bin/php", argv, envp); /* never returns on success - image is replaced */
    _exit(127);                       /* only reached if exec FAILED */
} else if (pid > 0) {
    int status;
    waitpid(pid, &status, 0);          /* reap the child; without this -> ZOMBIE */
    if (WIFEXITED(status)) printf("exit %d\n", WEXITSTATUS(status));
    if (WIFSIGNALED(status)) printf("signal %d\n", WTERMSIG(status));
}
```

Related calls: `vfork()` - legacy, child borrows parent's address space until exec
`clone()` - the real Linux syscall; `fork()` and `pthread_create()` are both thin wrappers over `clone()` with different `CLONE_*` sharing flags:
`CLONE_VM|CLONE_FILES|CLONE_FS|CLONE_SIGHAND|CLONE_THREAD` = thread
(none of those) = process
`posix_spawn()` - fork+exec in one call, avoids CoW cost for huge parents

Zombies and orphans

- **Zombie** (state Z) — the child has exited, but its exit status is still held in the process table because the parent has not called `wait()`. It consumes no memory or CPU, only a PID slot. Many zombies = a buggy parent. Fix the parent, or have it `signal(SIGCHLD, SIG_IGN)`. Killing a zombie does nothing — it is already dead; kill or restart the *parent*.
- **Orphan** — the parent died first. The child is re-parented to PID 1 (init/systemd), which reaps it correctly. Harmless.
- **PID 1 in a container** — if your app runs as PID 1 it inherits the reaping duty and does *not* get default signal handlers. That is why containers need `tini`/`--init`, and why `docker stop` sometimes appears to be ignored.

8.2 Process states



ps state codes: `ps -eo pid,stat,wchan:25,pcpu,comm | head -20`

GOTCHA | Linux load average counts **runnable + uninterruptible (D)** tasks, not just CPU demand. A load of 20 on a 4-core box with 5% CPU usage means processes are stuck waiting on I/O, not that the CPU is busy. Always read load average alongside `iowait`.

8.3 Signals

Signal	Default	Typical use
SIGTERM (15)	terminate	Polite shutdown. Catchable — this is what a queue worker traps to finish the current job before exiting.
SIGKILL (9)	terminate	Uncatchable, unblockable. The kernel removes the process. No cleanup runs, no destructors, no flushing. Last resort.
SIGINT (2)	terminate	Ctrl-C from the terminal.
SIGHUP (1)	terminate	Terminal hangup; by convention daemons reload their config on it (nginx, php-fpm).
SIGCHLD (17)	ignore	A child changed state — time to reap.
SIGSEGV (11)	core dump	Invalid memory access.
SIGSTOP/SIGCONT	stop/continue	Suspend and resume. SIGSTOP is also uncatchable.
SIGUSR1/USR2	terminate	Application-defined. PHP-FPM uses USR2 to reload, USR1 to reopen logs.

```
kill -TERM <pid>          kill -l                # list all signals
pkill -f 'queue:work'     killall -u www-data php-fpm8.4
timeout 30s ./slow_cmd    # SIGTERM after 30s;  timeout -s KILL 30s  to be brutal

# Graceful shutdown in Laravel: the worker checks a flag set by its SIGTERM handler
php artisan queue:restart  # sets a cache flag; workers exit after the current job
php artisan down --render=... # maintenance mode during deploy
systemctl reload php8.4-fpm # SIGUSR2: reload config without dropping connections
```

8.4 /proc — the kernel as a filesystem

```
/proc/<pid>/
  cmdline    the full argv (NUL-separated):  tr '\0' ' ' < /proc/<pid>/cmdline
  status     human-readable: State, Threads, VmRSS, VmHWM, VmSwap, voluntary_ctxt_switches
  stat       machine-readable: utime, stime, priority, nice, num_threads, starttime
  maps       every VMA: address range, permissions, backing file
  smaps_rollback Rss/Pss/Private_Dirty/Swap totals
  fd/        symlinks to every open file descriptor (ls -l /proc/<pid>/fd | wc -l)
  task/      one directory per THREAD (TID) -> ls /proc/<pid>/task | wc -l
  limits     the effective ulimits
  environ    the environment it was started with
  wchan      the kernel function it is currently sleeping in
  io         read_bytes / write_bytes actually issued to storage

/proc/stat    ctxt = total context switches since boot;  processes = total forks
/proc/loadavg 1m 5m 15m  running/total  lastpid
/proc/meminfo /proc/cpuinfo /proc/interrupts /proc/pressure/{cpu,io,memory} # PSI - very useful
```

8.5 Context switching — the mechanics and the cost

A **context switch** is the act of saving the state of the currently running task and restoring the state of another, so the CPU can be shared. It is pure overhead: no user work happens during it.

WHAT GETS SAVED / RESTORED

1. CPU registers: general purpose, program counter (RIP), stack pointer (RSP), flags
2. FPU / SIMD state (often lazily, only if the task used it)
3. Kernel stack pointer, thread-local base (FS/GS)
4. Memory context - ONLY IF SWITCHING BETWEEN PROCESSES:
load new CR3 (page table root) -> TLB entries become invalid
(PCID/ASID tagging avoids the full flush on modern CPUs)

THE COSTS, roughly

direct : 1-3 microseconds - saving/restoring registers, scheduler bookkeeping
 INDIRECT : usually far larger - cold L1/L2 caches, TLB misses, branch predictor pollution. The new task runs slowly for tens of microseconds until its working set is back in cache. This is the part people forget.

thread <-> thread, same process : cheaper (same CR3, TLB and much of the cache stay warm)
 process <-> process : more expensive (address-space switch)
 user <-> kernel (a syscall) : this is a MODE switch, NOT a context switch.
 ~50-300 ns. No scheduler involvement, same task.

MENTAL MODEL | Mode switch ≠ context switch. A syscall is a privilege-level change within the same task — cheap. A context switch changes which task is running — expensive. Getting this distinction right is a reliable interview differentiator.

Voluntary vs involuntary

- **Voluntary** — the task gave up the CPU itself: it blocked on I/O, waited on a lock, called `sleep()` or `yield()`. High voluntary counts point to an I/O-bound or lock-heavy workload.
- **Involuntary** — the scheduler preempted it: quantum expired, or a higher-priority task woke up. High involuntary counts point to CPU oversubscription — too many runnable threads for the cores available.

```
grep ctxt /proc/<pid>/status      # voluntary_ctxt_switches / nonvoluntary_ctxt_switches
pidstat -w -p <pid> 1           # cswch/s (voluntary) and ncswch/s (involuntary) per second
vmstat 1                        # 'cs' column = system-wide switches/sec; 'in' = interrupts
perf stat -e context-switches,cpu-migrations,page-faults -p <pid> -- sleep 10
perf sched record -- sleep 5 && perf sched latency      # per-task scheduling latency
cat /proc/pressure/cpu          # PSI: how much time tasks spend STALLED waiting for CPU
```

Rules of thumb:

- a few thousand cs/sec per core is normal for a busy server
- tens of thousands with low throughput = oversubscription or lock thrashing
- the fix is usually FEWER workers, not more: workers ~ cores for CPU-bound, workers >> cores only for I/O-bound work

Sizing PHP-FPM or Horizon by measurement rather than by guess is exactly this: increase workers until involuntary switches climb and throughput stops improving, then back off.

8.6 Namespaces and cgroups — how containers are built

- **Namespaces** virtualise *what a process can see*: pid (its own PID 1), net (its own interfaces), mnt (its own filesystem tree), uts (hostname), ipc, user (uid mapping), cgroup.
- **cgroups v2** limit *what a process can use*: `cpu.max`, `memory.max`, `io.max`, `pids.max`.
- A container is just a normal Linux process with a set of namespaces and a cgroup. There is no "container" object in the kernel — which is why `ps aux` on the host shows every container's processes.

```
lsns                                # list namespaces and their processes
systemd-cgls ; systemd-cgtop       # cgroup tree and live resource usage
cat /proc/<pid>/cgroup              # which cgroup a process belongs to
unshare --fork --pid --mount-proc bash # make your own PID namespace in one line
nsenter -t <pid> -n -m -p bash     # enter another process's namespaces (debugging containers)
```

9. Practice Lab

Do these in order on any Linux box (your Ubuntu desktop is fine). Each is written so you can see the concept happen rather than take it on faith. Estimated time in brackets.

Lab 1 — Prove that a race condition is real [15 min]

```
# race.c
#include <stdio.h>
#include <pthread.h>
#define N 8
#define ITERS 1000000
long counter = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *unsafe(void *a) { for (int i=0;i<ITERS;i++) counter++; return NULL; }
void *safe(void *a)   { for (int i=0;i<ITERS;i++) { pthread_mutex_lock(&m);
                                                    counter++;
                                                    pthread_mutex_unlock(&m);} return NULL; }

int main(int argc, char **argv) {
    pthread_t t[N];
    void *(*fn)(void*) = (argc>1 && argv[1][0]=='s') ? safe : unsafe;
    for (int i=0;i<N;i++) pthread_create(&t[i], NULL, fn, NULL);
    for (int i=0;i<N;i++) pthread_join(t[i], NULL);
    printf("expected %d got %ld\n", N*ITERS, counter);
    return 0;
}

gcc -O2 -pthread race.c -o race
./race          # unsafe -> a DIFFERENT wrong number every run
./race safe     # correct -> but time it: how much did the mutex cost?
time ./race ; time ./race safe

# Extend: replace the mutex with __atomic_fetch_add(&counter,1,__ATOMIC_RELAXED).
#          Is it faster than the mutex? Why?
# Extend: run under valgrind --tool=helgrind ./race to see the race reported automatically.
```

Lab 2 — Build a deadlock, then fix it with lock ordering [20 min]

```
# deadlock.c (compile: gcc -pthread deadlock.c -o dl)
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER, B = PTHREAD_MUTEX_INITIALIZER;

void *t1(void *) { pthread_mutex_lock(&A); sleep(1); pthread_mutex_lock(&B);
                  puts("t1 got both"); pthread_mutex_unlock(&B); pthread_mutex_unlock(&A);
                  return NULL; }

void *t2(void *) { pthread_mutex_lock(&B); sleep(1); pthread_mutex_lock(&A); /* <-- swap me */
                  puts("t2 got both"); pthread_mutex_unlock(&A); pthread_mutex_unlock(&B);
                  return NULL; }
```

Run it -> it hangs. Now diagnose it like production:

```
ps -o pid,stat,wchan:30 -p $(pgrep dl)
gdb -p $(pgrep dl) -batch -ex "thread apply all bt" # both stuck in pthread_mutex_lock
```

Then fix: make t2 lock A before B. Confirm it completes every time.

```
# MySQL version - open two terminals, run in opposite order, watch ERROR 1213:
T1: BEGIN; UPDATE seats SET status='x' WHERE id=1; -- then wait
T2: BEGIN; UPDATE seats SET status='x' WHERE id=2; -- then wait
T1: UPDATE seats SET status='x' WHERE id=2; -- blocks
T2: UPDATE seats SET status='x' WHERE id=1; -- DEADLOCK, one is rolled back
Then: SHOW ENGINE INNODB STATUS\G -> read the LATEST DETECTED DEADLOCK section
```

Lab 3 — Watch the scheduler make decisions [20 min]

```

nproc                                # how many cores are you working with?

# 1. Oversubscribe deliberately and watch involuntary switches climb
for i in $(seq 1 $(( $(nproc) * 4 ))); do (while ;; do ;; done) & done
vmstat 1 5                          # look at 'r' (run queue) and 'cs'
pidstat -w 1 3                      # nvcschw/s should be high
uptime                              # load average ~ 4x core count
kill %1 %2 %3 ... ; jobs           # (or: kill $(jobs -p) )

# 2. Prove that nice actually works
(while ;; do ;; done) &             P1=$!
nice -n 19 bash -c 'while ;; do ;; done' & P2=$!
taskset -pc 0 $P1 ; taskset -pc 0 $P2 # force BOTH onto core 0 so they must compete
pidstat -p $P1,$P2 1 5              # %CPU should be roughly 95 vs 5
kill $P1 $P2

# 3. Compare scheduling policies
chrt -p $$                          # your shell's policy (SCHED_OTHER, prio 0)
chrt -o -p 0 $$                     # explicitly set SCHED_OTHER

```

Lab 4 — Make page faults and thrashing visible [25 min]

```

# Minor vs major faults on the same file
python3 -c "open('/tmp/big','wb').write(b'x'*(512*1024*1024))"
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches' # cold cache
/usr/bin/time -v cat /tmp/big > /dev/null      # note "Major (requiring I/O) page faults"
/usr/bin/time -v cat /tmp/big > /dev/null      # run AGAIN - now warm: majors ~0, minors high

# Watch demand paging on a big allocation
python3 - <<'PY'
import os, time
print("pid", os.getpid()); input("check VmRSS in another shell, then press enter")
buf = bytearray(1_000_000_000) # allocated (VSZ jumps) but barely resident yet
input("VSZ jumped, RSS did not. Now press enter to TOUCH every page")
for i in range(0, len(buf), 4096): buf[i] = 1 # now RSS climbs - demand paging in action
input("compare VmRSS again")
PY
# in another shell: watch -n1 "grep -E 'VmSize|VmRSS' /proc/<pid>/status"

# System-wide paging behaviour
vmstat 1 # si/so must stay at 0 on a healthy box
sar -B 1 5 # majflt/s, pgscank/s, pgsteal/s
cat /proc/pressure/memory # 'some'/'full' avg10 > 0 means real memory pressure

```

Lab 5 — fork, exec, zombies and CoW [20 min]

```

# Create a zombie on purpose
python3 - <<'PY'
import os, time
pid = os.fork()
if pid == 0: os._exit(0) # child exits immediately
print("parent", os.getpid(), "child", pid, "- NOT calling wait()")
time.sleep(60) # child is now a zombie
PY
# elsewhere: ps -eo pid,ppid,stat,comm | grep -E ' Z |defunct'
# then let the parent exit -> the zombie is reparented to systemd and reaped instantly.

# Copy-on-write in action
python3 - <<'PY'
import os
data = bytearray(200*1024*1024) # 200 MB, touched
for i in range(0, len(data), 4096): data[i] = 1
pid = os.fork()
print(("child " if pid==0 else "parent "), os.getpid())
input() # compare RSS of both in another shell - they SHARE the pages
PY
# Sum RSS of both -> looks like 400 MB. Now check PSS instead:
# smem -k -P python3 or grep Pss /proc/<pid>/smaps_rollup

```

```
# PSS shows the truth: ~200 MB total, shared. THIS is why summing RSS over FPM workers lies.

# Trace what a real process actually asks the kernel for
strace -f -e trace=process php artisan --version 2>&1 | head -40
strace -c -p $(pgrep -n php-fpm)      # syscall histogram of a live worker (Ctrl-C to stop)
ltrace ./prog                        # library calls instead of syscalls
```

Lab 6 — Measure context-switch cost yourself [30 min]

```
# Ping-pong two threads through a pipe on ONE core; each round trip = 2 context switches
# pingpong.c : two threads, pipe A->B and B->A, 100000 round trips, taskset to core 0
taskset -c 0 ./pingpong 100000
#   time / (2 * iterations) ~= the cost of one context switch (expect ~2-8 microseconds)

# Compare against a pure mode switch (syscall, no scheduling):
taskset -c 0 perf stat -e syscalls:sys_enter_getpid \
    python3 -c "import os; [os.getpid() for _ in range(1000000)]"
#   getpid is ~100x cheaper than a context switch. That gap is the whole lesson.

# Then measure your real workload:
perf stat -e context-switches,cpu-migrations,page-faults,task-clock \
    -p $(pgrep -n php-fpm) -- sleep 20
#   Question to answer for yourself: for YOUR queue workload, are the switches
#   mostly voluntary (I/O-bound -> add workers) or involuntary (CPU-bound -> stop adding)?
```

Lab 7 — Apply it to your own stack [1 hour]

- 1 Take a running PHP-FPM pool. Record `pm.max_children`, then measure average worker PSS with `smem -k -P php-fpm`. Is `max_children × PSS` under your RAM budget? Recalculate the correct value and justify it in two sentences.
- 2 Run `pidstat -w 1` against a busy queue worker for 60 seconds. Classify it as I/O-bound or CPU-bound from the voluntary/involuntary ratio, then predict whether adding workers will help. Test the prediction.
- 3 Find one place in your codebase with a check-then-act race (a `firstOrCreate` without a unique index, a balance update, a seat or ticket assignment). Write down which of the fixes from Section 2.1(b) applies and why.
- 4 Audit one multi-table transaction for lock ordering. Does every code path touch the tables and rows in the same order? If not, you have a latent deadlock.
- 5 Set `nice 10` on a reporting/export worker and confirm with `pidstat` that request-serving latency improves under load.

10. Rapid Revision and Interview Q&A

10.1 One-line answers you should be able to give instantly

Question	Answer
Process vs thread?	A process owns an address space; threads share one. Threads share heap, globals and file descriptors; each has its own stack and registers.
Concurrency vs parallelism?	Concurrency = dealing with many things at once (structure). Parallelism = doing many things at once (execution).
Mutex vs semaphore?	A mutex has an owner and provides mutual exclusion; a counting semaphore has no owner and counts N permits, used for signalling and resource pools.
Mutex vs spinlock?	Mutex sleeps (context switch, good for long holds); spinlock busy-waits (no switch, only for very short holds on multi-core).
Four deadlock conditions?	Mutual exclusion, hold-and-wait, no preemption, circular wait. All four required; break any one.
Deadlock vs livelock vs starvation?	Deadlock: blocked forever in a cycle. Livelock: active but making no progress. Starvation: perpetually outranked.
Why is SJF not used in practice?	Burst times are unknown in advance and long jobs starve. MLFQ/CFS approximate it from observed behaviour.
What does CFS optimise?	Proportional fairness: it always runs the task with the smallest weighted virtual runtime.
Page fault: minor vs major?	Minor = the page is already in RAM, only the mapping was missing (microseconds). Major = it must be read from disk or swap (milliseconds).
Why multi-level page tables?	A flat 48-bit table would need hundreds of GB per process; a hierarchy only materialises the parts actually used.
What is the TLB?	A hardware cache of virtual→physical translations. Without it every access would cost 4–5 memory reads.
Thrashing?	Working sets exceed physical memory, so the system spends more time paging than executing. Fix: fewer concurrent processes or more RAM.
Internal vs external fragmentation?	Internal = waste inside an allocated block (rounding). External = free memory exists but is non-contiguous.
Context switch vs mode switch?	Context switch changes which task runs (microseconds, plus cache/TLB damage). Mode switch is a syscall crossing into the kernel in the same task (nanoseconds).
Copy-on-write?	After fork, parent and child share pages read-only; the first write faults and copies just that page. Makes fork cheap.
Zombie process?	An exited child whose parent has not reaped it. It holds a PID slot and an exit status. Fix the parent.
What does load average measure?	Runnable <i>plus</i> uninterruptible (D-state) tasks — so it reflects I/O waiting too, not just CPU demand.
Why is Redis single-threaded?	It eliminates locking entirely: every command is atomic by construction, and the bottleneck is network/memory, not CPU.

10.2 The connections worth being able to draw

```

Threads share memory -----> races -----> need locks -----> risk deadlock
                                     |
                                     costs contention   fix by lock ordering
                                     |
                                     limits speedup (Amdahl)

Virtual memory ---> page tables ---> slow ---> TLB ---> misses hurt ---> huge pages
    |
    |               |
    |               +---> permission bits --> CoW, shared libs, NX, guard pages
    |
    +---> demand paging --> page faults --> too many --> thrashing --> "fewer workers"

Scheduler picks a task ---> context switch ---> cache/TLB go cold ---> real cost is INDIRECT
                                     |
                                     voluntary (I/O bound: add workers) vs involuntary (CPU bound: stop adding)

```

10.3 The production instincts to keep

- **Worker count is not a guess.** CPU-bound → roughly one worker per core. I/O-bound → many more, bounded by memory ($PSS \times \text{workers}$) and by downstream connection limits, not by cores.
- **Low CPU + unresponsive** = blocked on I/O, thrashing, deadlock, or cgroup throttling. **High CPU + no progress** = livelock, spinning, or a busy loop.
- **Never hold a lock across I/O.** Not a DB query, not an HTTP call, not a file write.
- **Lock ordering is free insurance.** Sort your IDs before you lock them.
- **Short transactions.** Every millisecond a transaction is open is a millisecond of lock contention and deadlock probability.
- **Idempotency beats perfect locking** in distributed systems. TTLs expire, networks partition, workers get SIGKILLED. Design so a repeat is harmless.
- **Measure with the right metric:** PSS not RSS, "available" not "free", major faults not total faults, involuntary switches not total switches.

10.4 Where to go deeper

- **Operating Systems: Three Easy Pieces** (Arpaci-Dusseau) — free online, and the single best fit for this note. Read the Virtualization and Concurrency parts.
- **The Linux Programming Interface** (Kerrisk) — the reference for everything in Section 8. Also `man 2 fork`, `man 7 signal`, `man 5 proc`.
- **Systems Performance** (Gregg) — the methodology for Sections 5–8 in production; pairs with his USE method.
- **Operating System Concepts** (Silberschatz) — the classic for exam-style algorithm questions (Banker's, page replacement, scheduling calculations).

End of note — Phase 2, Topic 4: Operating Systems. Next natural step: Networking, then Databases & Storage Engines, both of which reuse everything in Sections 3, 4 and 7.